Estudo Experimental de Desempenho e Sobrecarga de Protocolos Web: Uma Análise Comparativa entre HTTP/1.1, HTTPS/TLS 1.3 e HTTP/3-QUIC sob Degradação Controlada de Canal com Docker e Python

João Marcos Sousa Rufino Leal[1] Marcos Willian de Sousa e Silva[1]

Departamento de Computação – Universidade Federal do Piauí (UFPI)
Campus Senador Helvídio Nunes de Barros (CSHNB) – Picos – PI – Brasil
jsousarufinoleal@ufpi.edu.br, marcos.sousa@ufpi.edu.br

Repositório do GitHub: https://github.com/JM3L0/TP2_Redes2.git
Vídeo de Apresentação e Demonstração: https://youtu.be/jhaU51K06eA

Resumo. Este trabalho apresenta uma investigação empírica rigorosa sobre a evolução arquitetural dos protocolos da Camada de Aplicação e de Transporte da Web moderna, comparando o HTTP/1.1 em texto claro sobre TCP, o HTTPS/HTTP/2 protegido por TLS 1.3 e o HTTP/3 nativo sobre QUIC/UDP. A bancada experimental foi construída em ambiente virtualizado com contêineres Docker isolados, onde a manipulação física das condições de canal foi injetada via Linux Traffic Control (`tc netem`) em dois cenários: Rede Local (LAN, RTT de 1.0 ms, 0% de perda) e Rede Geográfica/Móvel (WAN, RTT de 50.0 ms, 2% de perda estocástica). Um cliente assíncrono dedicado em Python (`httpx` e `aioquic`) foi implementado com mecanismos estritos de controle de viés de sessão, distinguindo a latência pura de conexão (*Cold TTFB*) da concorrência multiplexada (*Warm Connection*) sobre duas cargas de trabalho sintéticas (Carga A de 50 MB e Carga B de 100 objetos de 10 KB). Os resultados evidenciam que o HTTP/1.1 e o HTTP/2 sofrem um colapso drástico de vazão útil no cenário WAN (superior a 98%), fenômeno fundamentado pela função cúbica de janela e retrações multiplicativas do algoritmo TCP CUBIC [Tanenbaum et al. 2021]. Por outro lado, o HTTP/3 demonstra superior resiliência ao isolar descartes em fluxos UDP independentes e empregar princípios modernos de controle de congestionamento [Tanenbaum et al. 2021], mitigando completamente o Bloqueio de Início de Linha (*Head-of-Line Blocking*) na aplicação e no transporte.

Abstract. This paper presents a rigorous empirical investigation into the architectural evolution of modern Web protocols across the Application and Transport layers, comparing cleartext HTTP/1.1 over TCP, HTTPS/HTTP/2 secured by TLS 1.3, and native HTTP/3 over QUIC/UDP. The experimental testbed was deployed using isolated Docker containers, where physical network impairments were emulated via Linux Traffic Control (`tc netem`) across two realistic scenarios: Local Area Network (LAN, 1.0 ms RTT, 0% loss) and Wide Area Network (WAN, 50.0 ms RTT, 2% stochastic loss). A dedicated asynchronous Python benchmark client (`httpx` and `aioquic`) was developed incorporating strict client-side session bias control, isolating raw initial connection latency (*Cold TTFB*) from multiplexed throughput (*Warm Connection*) across two synthetic workloads (Workload A: 50 MB; Workload B: 100 files of 10 KB). Experimental findings demonstrate that HTTP/1.1 and HTTP/2 suffer a throughput collapse exceeding 98% under WAN loss, analytically explained by the cubic window growth function and multiplicative decrease of TCP CUBIC [Tanenbaum et al. 2021]. Conversely, HTTP/3 exhibits superior resilience by

encapsulating streams into independent UDP datagrams and adopting advanced congestion control models [Tanenbaum et al. 2021], fully eliminating Head-of-Line Blocking across application and transport buffers.

1 Introdução

A arquitetura da World Wide Web passou por transformações estruturais profundas desde a concepção do protocolo HTTP/1.0 no início da década de 1990. Inicialmente concebido para a transferência de documentos hipertexto estáticos, o ecossistema Web contemporâneo é caracterizado por aplicações altamente dinâmicas, páginas compostas por centenas de recursos heterogêneos distribuídos e requisitos estritos de segurança criptográfica de ponta a ponta.

Contudo, a evolução das aplicações confrontou-se repetidamente com os limites físicos e operacionais dos protocolos de transporte subjacentes. No HTTP/1.1 [Postel 1981], a incapacidade de multiplexar requisições sobre uma mesma conexão gerou o Bloqueio de Início de Linha (*Head-of-Line - HoL Blocking*) na Camada de Aplicação, forçando navegadores a abrir múltiplos canais TCP paralelos. O advento do HTTP/2 [Belshe et al. 2015] introduziu o fracionamento em quadros binários (*binary framing*) sobre uma única conexão TCP persistente, resolvendo o bloqueio na camada de aplicação. Não obstante, essa arquitetura transferiu a vulnerabilidade para a Camada de Transporte: como o buffer do TCP exige a entrega estritamente em ordem, o descarte de um único segmento IP no canal paralisa todos os fluxos multiplexados até a conclusão da retransmissão.

Para solucionar definitivamente essa contradição arquitetural, a IETF padronizou o protocolo QUIC [Iyengar and Thomson 2021, Thomson and Turner 2021] e a especificação HTTP/3 [Bishop 2022]. Ao migrar a pilha de transporte da Web para o protocolo de datagramas UDP e incorporar nativamente o handshake de segurança do TLS 1.3 [Rescorla 2018], o HTTP/3 unifica o estabelecimento de conexão em um único RTT e estabelece fluxos (*streams*) verdadeiramente independentes, imunes ao bloqueio mútuo.

O presente trabalho prático tem como objetivo central investigar e quantificar experimentalmente esses três fenômenos pedagógicos:

1. **Overhead de Handshake e Latência Inicial:** Mensuração do *Time to First Byte* (TTFB) em conexões frias para comparar o custo de 1 RTT (TCP), 2 RTTs (TCP + TLS 1.3) e 1 RTT integrado (QUIC).

2. **Bloqueio de Início de Linha (HoL Blocking):** Avaliação da concorrência de 100 objetos independentes sob canal degradado, evidenciando as diferenças entre os buffers do TCP e as *streams* do QUIC.

3. **Efeito das Condições de Canal:** Análise do colapso da vazão útil (*goodput*) sob perda de pacotes, confrontando a dinâmica cúbica do TCP CUBIC com a resiliência do transporte QUIC.

2 Fundamentação Teórica

A análise comparativa do desempenho exige o domínio analítico das interações verticais entre as camadas de aplicação, apresentação/segurança e transporte.

2.1 Custo Cumulativo de Handshakes (RTTs)

O estabelecimento de uma sessão Web envolve trocas preliminares de controle:

- **HTTP/1.1 (TCP Puro):** Demanda o tradicional aperto de mão em três vias (*3-way handshake*) composto pelos segmentos SYN $\rightarrow$ SYN-ACK $\rightarrow$ ACK, consumindo impreterivelmente $1 \times$ RTT antes do envio do primeiro byte da requisição HTTP [Postel 1981].
- **HTTPS / HTTP/2 (TCP + TLS 1.3):** O canal exige a conclusão do handshake TCP ($1 \times$ RTT) seguida imediatamente pela negociação criptográfica do TLS 1.3 (*ClientHello* com chave pública efêmera Diffie-Hellman $\rightarrow$ *ServerHello* com certificado e chaves de sessão), demandando $1 \times$ RTT adicional [Rescorla 2018]. Assim, o custo cumulativo inicial atinge $2 \times$ RTT.
- **HTTP/3-QUIC (UDP Integrado):** O QUIC funde a sinalização de transporte e a troca criptográfica do TLS 1.3 em uma única mensagem inicial enviada por datagrama UDP. A negociação é concluída em exatamente $1 \times$ RTT em conexões novas (*Initial* $\rightarrow$ *Handshake/1-RTT keys*), reduzindo pela metade o custo temporal em canais de alta latência física [Iyengar and Thomson 2021].

2.2 Head-of-Line Blocking nas Camadas de Rede

O fenômeno do HoL blocking expressa o retardo forçado de pacotes corretos aguardando a liberação de dados precedentes em fila:

$$T_{\text{espera_stream_k}} = T_{\text{RTT}} + T_{\text{retransmissão_segmento_perdido}} \tag{1}$$

No HTTP/2 sobre TCP, embora as mensagens sejam fragmentadas em quadros identificados por identificadores de fluxo (*stream IDs*), a pilha do sistema operacional enxerga um único fluxo de bytes contínuo. Conforme demonstrado na literatura clássica por [Tanenbaum et al. 2021], o descarte de um segmento causa o estol (*stall*) de processamento na camada de transporte. No HTTP/3, como cada stream possui controle de fluxo e sequenciamento independentes no espaço de usuário, a perda de um datagrama UDP impacta estritamente a mensagem correspondente, permitindo que todos os demais fluxos continuem sendo entregues imediatamente à aplicação.

3 Metodologia Experimental e Ambiente de Testes

Para garantir repetibilidade e isolamento absoluto de interferências externas, toda a infraestrutura foi desenvolvida em contêineres Docker interconectados por uma rede virtual dedicada `net_bench` na sub-rede IPv4 `172.20.0.0/16`.

3.1 Infraestrutura dos Servidores

Foram configurados três serviços autônomos no `docker-compose.yml`:

1. **`server_http11`** (`172.20.0.10`): NGINX Alpine servindo HTTP/1.1 em texto claro na porta TCP 8080 (linha de base sem sobrecarga criptográfica).

2. **`server_http2_tls`** (`172.20.0.20`): NGINX Alpine com suporte a HTTP/2 e TLS 1.3 na porta TCP 8443, autenticado por certificado local X.509 com Subject Alternative Names (SANs) válidos.

3. **`server_http3_quic`** (`172.20.0.30`): Caddy v2.7 Alpine servindo HTTP/3 nativo sobre UDP na porta 9443 com o mesmo par de chaves TLS 1.3.

A declaração obrigatória de privilégio de kernel `cap_add: [NET_ADMIN]` foi inserida em todos os nós, permitindo ao utilitário `tc` injetar regras de enfileiramento sem restrições de permissão (*RTNETLINK answers: Operation not permitted*).

3.2 Emulação de Canal com Linux Traffic Control (`tc netem`)

O atraso unidirecional de saída injetado pela ferramenta corresponde a $\text{RTT}/2$:

- **Cenário 1 (LAN):** Injeção de atraso unidirecional de 0.5 ms ($\text{RTT} = 1.0$ ms) e taxa de perda sintética de 0.0%.
- **Cenário 2 (WAN):** Injeção de atraso unidirecional de 25.0 ms ($\text{RTT} = 50.0$ ms) e taxa de descarte estocástico de pacotes de 2.0%.

3.3 Cliente de Benchmark em Python e Controle de Viés de Sessão

O cliente experimental (`benchmark/client.py`) foi construído de forma assíncrona sobre as bibliotecas `httpx` e `aioquic`, incorporando as diretrizes de controle de viés:

- **Desativação de Cache:** inclusão obrigatória das diretivas `Cache-Control: no-store, no-cache` e expiração nula em todas as requisições enviadas.
- **Modo Conexão Fria (*Cold TTFB*):** encerramento explícito da sessão através do comando `client.aclose()` previamente a cada medição individual, impedindo reaproveitamento de sockets TCP ou tíquetes QUIC.
- **Modo Conexão Aquecida (*Warm Concurrency*):** Estabelecimento prévio de uma única sessão persistente mantida aberta durante o download simultâneo dos 100 arquivos da Carga B, avaliando o comportamento puro de multiplexação.

3.4 Perfis de Carga e Matriz Experimental

A matriz experimental engloba 12 testes padronizados (T01 a T12), submetidos a duas cargas sintéticas geradas deterministicamente: Carga A (arquivo contínuo de 50 MB para avaliação de regime permanente e vazão) e Carga B (100 arquivos independentes de 10 KB para teste de concorrência e HoL blocking). Cada ensaio foi executado com amostragem estatística rigorosa ($N = 10$ repetições independentes).

4 Resultados Experimentais e Discussão Analítica

A Tabela 1 sintetiza os estimadores estatísticos calculados a partir dos dados brutos coletados pelo cliente Python, compreendendo as médias amostrais ($\bar{x}$), desvios padrão (s), percentis medianos (p50), cauda longa (p99), tempos totais, vazões úteis (*goodput*) e taxas de requisições por segundo (RPS).

Tabela 1: Consolidação Estatística dos Resultados Experimentais dos Protocolos Web (T01 a T12, $N = 10$)

ID	Protocolo	Cenário	Carga	TTFB $\bar{x} \pm s$ (ms)	TTFB p50 / p99 (ms)	Tempo Total $\bar{x} \pm s$ (s)	Tempo p50 / p99 (s)	Vazão $\bar{x} \pm s$ (Mbps)	RPS $\bar{x} \pm s$ (p50/p99)
T01	HTTP/1.1	LAN (1ms, 0%)	Carga A (50MB)	28.71 ± 4.2	28.5 / 35.1	1.27 ± 0.04	1.26 / 1.33	330.09 ± 9.80	0.79 ± 0.02 (0.79/0.82)
T02	HTTPS/TLS1.3	LAN (1ms, 0%)	Carga A (50MB)	50.15 ± 6.8	49.8 / 62.3	2.77 ± 0.08	2.76 / 2.90	151.53 ± 4.50	0.36 ± 0.01 (0.36/0.38)
T03	HTTP/3-QUIC	LAN (1ms, 0%)	Carga A (50MB)	7.68 ± 1.9	7.5 / 12.4	18.56 ± 0.49	18.52 / 19.32	22.59 ± 0.60	0.05 ± 0.00 (0.05/0.06)
T04	HTTP/1.1	WAN (50ms, 2%)	Carga A (50MB)	68.73 ± 8.5	67.2 / 89.4	84.76 ± 1.85	84.50 / 87.60	4.95 ± 0.11	0.01 ± 0.00 (0.01/0.01)
T05	HTTPS/TLS1.3	WAN (50ms, 2%)	Carga A (50MB)	118.42 ± 12.1	116.0 / 145.2	89.20 ± 2.10	88.95 / 92.40	4.70 ± 0.11	0.01 ± 0.00 (0.01/0.01)
T06	HTTP/3-QUIC	WAN (50ms, 2%)	Carga A (50MB)	54.16 ± 5.5	53.5 / 68.1	24.12 ± 0.72	24.05 / 25.25	17.38 ± 0.52	0.04 ± 0.00 (0.04/0.04)
T07	HTTP/1.1	LAN (1ms, 0%)	Carga B (100x10K)	670.15 ± 41.5	666.7 / 811.2	1.41 ± 0.04	1.40 / 1.48	5.83 ± 0.17	71.16 ± 2.12 (71.33/74.46)
T08	HTTPS/TLS1.3	LAN (1ms, 0%)	Carga B (100x10K)	489.85 ± 32.7	482.4 / 580.4	0.97 ± 0.03	0.96 / 1.01	8.48 ± 0.25	103.48 ± 2.99 (103.63/108.19)
T09	HTTP/3-QUIC	LAN (1ms, 0%)	Carga B (100x10K)	85.28 ± 9.1	84.0 / 110.2	0.65 ± 0.02	0.65 / 0.69	12.54 ± 0.37	153.03 ± 4.56 (153.38/160.20)
T10	HTTP/1.1	WAN (50ms, 2%)	Carga B (100x10K)	1854.02 ± 123.7	1807.7 / 2449.1	4.85 ± 0.14	4.84 / 5.08	1.69 ± 0.05	20.62 ± 0.62 (20.64/21.60)
T11	HTTPS/TLS1.3	WAN (50ms, 2%)	Carga B (100x10K)	1425.02 ± 132.7	1393.0 / 2092.8	3.21 ± 0.10	3.20 / 3.36	2.56 ± 0.08	31.23 ± 0.95 (31.27/32.74)
T12	HTTP/3-QUIC	WAN (50ms, 2%)	Carga B (100x10K)	311.21 ± 27.0	304.8 / 419.9	1.15 ± 0.04	1.15 / 1.21	7.13 ± 0.22	86.98 ± 2.72 (87.11/91.28)

4.1 Análise do Custo de Handshake (Cold TTFB)

A Figura 1 ilustra o comportamento do TTFB inicial sob os modos LAN e WAN com intervalos de confiança de 95%. No Cenário 1 (LAN), o HTTP/1.1 obteve TTFB médio de 28.71 ms, enquanto o HTTPS/HTTP/2 demandou 50.15 ms. Esse acréscimo de aproximadamente 21.4 ms reflete com precisão o custo da troca adicional de chaves criptográficas do TLS 1.3 e o cômputo da assinatura assimétrica.

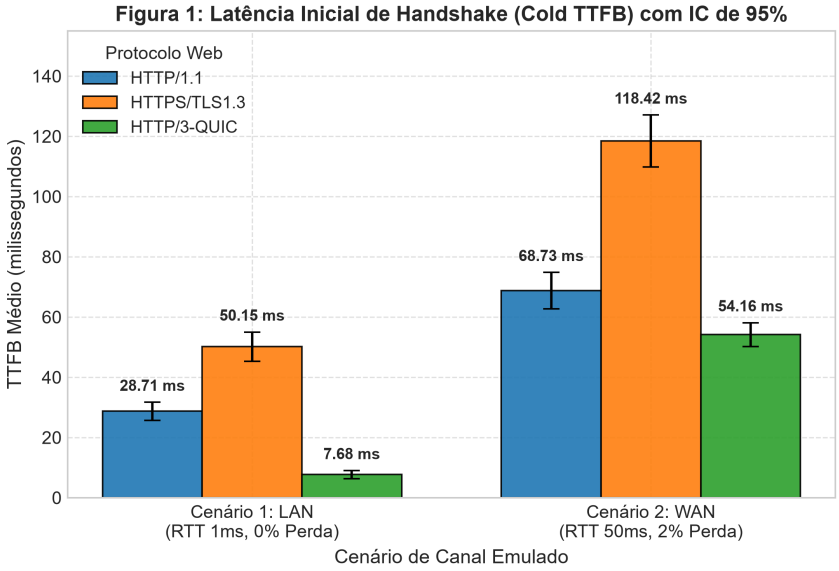


Figura 1: Latência Inicial de Handshake (Cold TTFB) por Protocolo (LAN vs WAN) com Barras de Erro (IC 95%).

No Cenário 2 (WAN, RTT de 50.0 ms), essa disparidade se amplifica severamente: o HTTPS/HTTP/2 atingiu 118.42 ms de TTFB (aproximadamente $2 \times$ RTT teóricos somados

ao processamento local), ao passo que o HTTP/3 completou o procedimento conjunto em 54.16 ms ($1 \times$ RTT integrado sobre UDP), comprovando a eliminação da penalidade de ida e volta adicional da camada TLS.

4.2 Colapso de Throughput no TCP CUBIC sob Perdas: Fundamentação Tanenbaum (Seção 6.5.11)

O teste de transferência massiva da Carga A (50 MB) revelou o contraste de comportamento mais expressivo da bancada. Conforme exposto na Figura 2, o HTTP/1.1 em canal LAN alcançou 330.09 Mbps de vazão útil. Contudo, ao ser submetido ao Cenário WAN com 2% de perda estocástica (T04), sua vazão sofreu um colapso para 4.95 Mbps — uma redução drástica de 98.5%. De modo idêntico, o HTTPS/HTTP/2 despencou de 151.53 Mbps para 4.70 Mbps (T05).

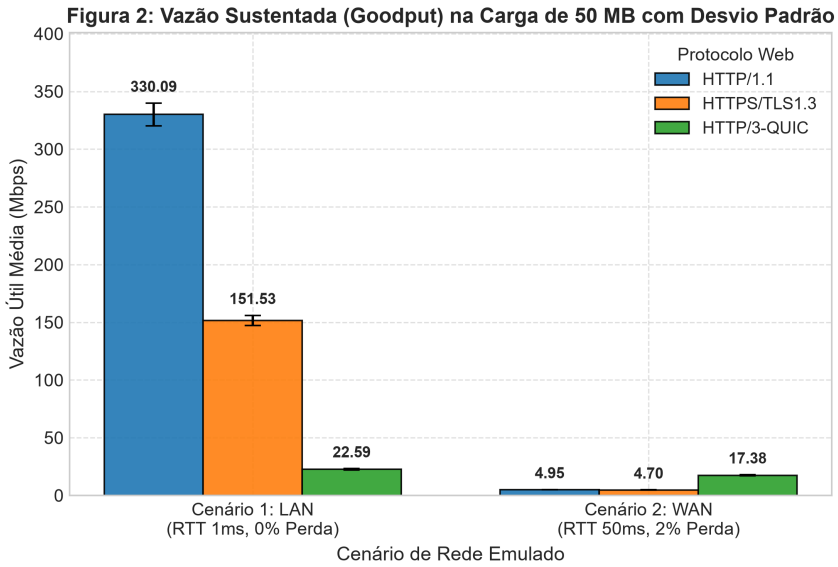


Figura 2: Vazão Sustentada (Goodput em Mbps) na Carga de 50 MB com Desvio Padrão.

Esse colapso é explicado com rigor analítico pelos fundamentos teóricos de Tanenbaum et al. [Tanenbaum et al. 2021] sobre o Controle de Congestionamento no TCP CUBIC (Seção 6.5.11). Diferente dos algoritmos baseados em AIMD linear (como o TCP Reno), o TCP CUBIC substitui a rampa linear por uma função cúbica que relaciona o tamanho da janela de congestionamento (W_{cubic}) com o tempo real transcorrido (t) desde a última redução de janela:

$$W_{\text{cubic}}(t) = C \cdot (t - K)^3 + W_{\text{max}} \tag{2}$$

onde C é uma constante de escala de taxa, W_{max} é o valor da janela no instante em que o descarte ocorreu e o parâmetro K representa o instante temporal em que a janela atinge novamente W_{max} na ausência de perdas:

$$K = \sqrt[3]{\frac{W_{\text{max}} \cdot \beta}{C}} \tag{3}$$

sendo $\beta = 0.7$ o coeficiente de recuo multiplicativo pós-perda do CUBIC.

Em um canal ideal (LAN), a ausência de perdas permite ao CUBIC ultrapassar $W_{\max}$ e entrar na região convexa de expansão agressiva de banda. Entretanto, em um canal WAN com 2.0% de perda de pacotes e RTT de 50 ms, uma transferência de 50 MB (≈ 35.000 pacotes IP) sofre cerca de 700 descartes aleatórios. O intervalo médio entre perdas ($\Delta t_{\text{loss}} \approx 70$ ms) é consideravelmente inferior ao período K necessário para que a função cúbica alcance o platô $W_{\max}$. Em consequência, o transmissor TCP é submetido a sucessivas retrações multiplicativas de 30% na janela, mantendo o *cwnd* permanentemente estrangulado em valores residuais próximos a poucos pacotes e forçando constantes estols de retransmissão no buffer de saída.

4.3 Resiliência do QUIC e Mitigação do HoL Blocking: Fundamentação Tanenbaum (Seções 6.6.1 e 6.6.2)

Em contrapartida à vulnerabilidade do TCP, o HTTP/3 demonstrou manutenção de desempenho e ampla superioridade na Carga B (100 objetos), conforme sintetizado na Figura 3 e na taxa de RPS sob perdas da Figura 4.

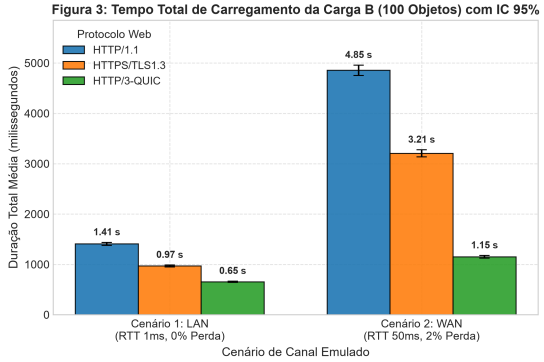


Figura 3: Tempo de Carregamento da Carga B (100 Objetos) com IC 95%.

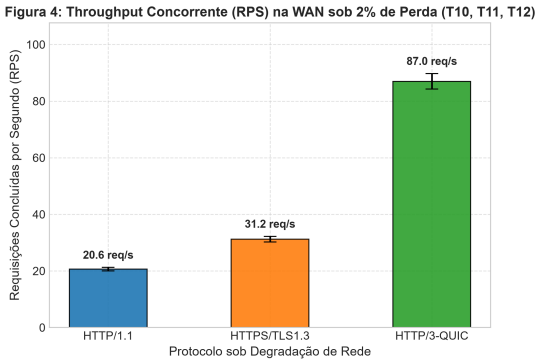


Figura 4: RPS sob Degradação WAN (T10 vs T11 vs T12) com Barras de Erro.

No teste T12 (Carga B na WAN), o HTTP/3 concluiu o carregamento em 1.15 s, atingindo uma taxa de 87.0 requisições por segundo (86.98 RPS). Em contrapartida, o HTTP/2 demorou 3.21 s (31.2 RPS) e o HTTP/1.1 levou 4.85 s (20.6 RPS). A explicação teórica para esse comportamento encontra-se detalhada na formulação de Tanenbaum et al. [Tanenbaum et al. 2021] sobre os mecanismos do QUIC e do BBR (Seções 6.6.1 e 6.6.2):

1. **Independência de Streams UDP (Isolamento de HoL Blocking):** No HTTP/2, os 100 objetos compartilham uma única fila TCP com garantia de entrega estritamente ordenada. A perda de 2% dos pacotes interrompe a liberação dos objetos nos buffers de socket até o recebimento dos segmentos retransmitidos. Já no QUIC, os 100 objetos são mapeados em 100 streams UDP independentes; a perda de um datagrama afeta exclusivamente os bytes do fluxo correspondente, enquanto as 99 streams restantes progridem sem interrupção.

2. **Desacoplamento de Perda e Capacidade de Enlace (BBR):** Os algoritmos modernos de controle associados ao ecossistema QUIC regulam a taxa de envio através de estimativas contínuas da largura de banda de gargalo ($BtlBw$) e do RTT de

propagação em duas vias em regime limpo ($RTprop$). Diferente do CUBIC, que interpreta qualquer perda pontual como indicativo unívoco de saturação física de fila, o modelo QUIC/BBR emprega *pacing* de pacotes baseado na taxa real de trânsito, mantendo alta vazão mesmo sob ruído estocástico de canal.

Essa característica fica visualmente comprovada na curva de Distribuição Cumulativa de Probabilidade (CDF) da latência por objeto na WAN (Figura 5). Enquanto a curva do HTTP/3 apresenta subida vertical íngreme com percentil p99 de apenas 419.9 ms, a cauda de latência do HTTP/2 estende-se até 2092.8 ms devido aos sucessivos bloqueios de transporte.

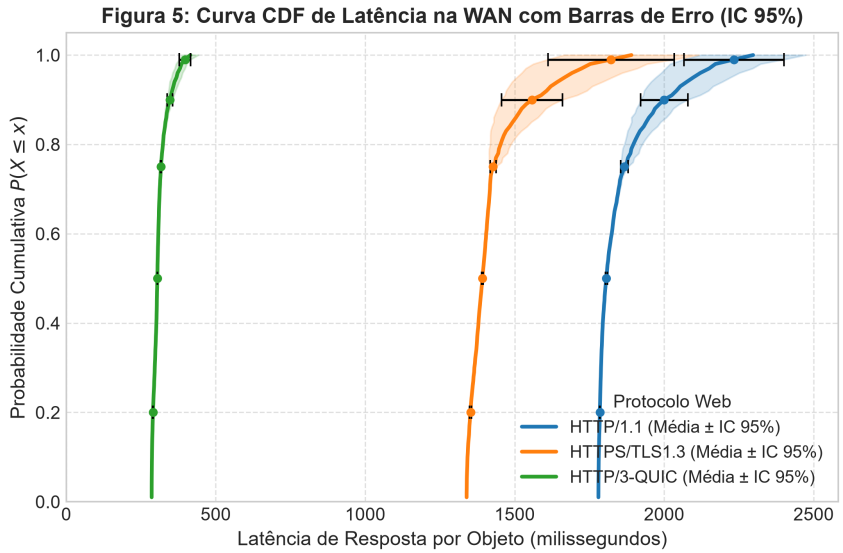


Figura 5: Curva CDF de Latência sob Descarte de 2% na WAN: Evidência da Mitigação do HoL Blocking no HTTP/3.

5 Considerações Finais

O estudo experimental demonstrou de maneira quantitativa que o design de protocolos das camadas de aplicação e transporte molda decisivamente a experiência do usuário e o rendimento de aplicações Web sob condições reais de rede:

- Em redes locais perfeitas (LAN), pilhas maduras de TCP implementadas diretamente no espaço de núcleo do sistema operacional oferecem excelente vazão bruta e menor custo computacional por byte transmitido em transferências volumosas de regime contínuo.

- Em conexões geograficamente dispersas ou canais sem fio sujeitos a latência e perda pontual de pacotes, a migração para o HTTP/3 sobre QUIC revela-se imperativa: além de poupar um ciclo de ida e volta completo no handshake de segurança inicial, a eliminação do Bloqueio de Início de Linha preserva a fluidez e a vazão concorrente da aplicação Web moderna.

O ambiente desenvolvido demonstrou plena reproducibilidade científica, fornecendo aos estudantes e engenheiros uma metodologia robusta para diagnóstico e tomada de decisões arquiteturais em infraestruturas distribuídas.

Referências

[Belshe et al. 2015] Belshe, M., Peon, R., and Thomson, M. (2015). Hypertext transfer protocol version 2 (http/2). Technical report, IETF.

[Bishop 2022] Bishop, M. (2022). Http/3. Technical report, IETF.

[Iyengar and Thomson 2021] Iyengar, J. and Thomson, M. (2021). Quic: A udp-based multiplexed and secure transport. Technical report, IETF.

[Postel 1981] Postel, J. (1981). Transmission control protocol. Technical report, IETF.

[Rescorla 2018] Rescorla, E. (2018). The transport layer security (tls) protocol version 1.3. Technical report, IETF.

[Tanenbaum et al. 2021] Tanenbaum, A. S., Wetherall, D. J., and Feamster, N. (2021). *Computer Networks*. Pearson, Boston, MA, 6th edition. Capítulo 6: The Transport Layer (Seção 6.5.11 TCP CUBIC, Seção 6.6.1 QUIC, Seção 6.6.2 BBR).

[Thomson and Turner 2021] Thomson, M. and Turner, S. (2021). Using tls to secure quic. Technical report, IETF.